LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

05

Linux Process and Service Management

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 What Is a Process?
02 Parent and Child Processes
03 Process States
04 Monitoring Processes
05 kill and Signals
06 Foreground and Background
07 Understanding systemd
08 Managing Services: systemctl
09 systemd Service Files
10 CPU and Memory Monitoring
11 Priority and Resource Limits
12 Cron and Scheduled Tasks
13 Service Logs: journalctl
14 Troubleshooting Scenarios
15 Interview Q&A
16 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux

github.com/rv-ops/linux-interview-preparation-guide

CHAPTER OBJECTIVE

Understand how Linux runs applications and background processes, how to monitor and control them, and how to manage services using systemd. Learn practical commands for troubleshooting high CPU usage, memory consumption, unresponsive processes and failed services.

1 What Is a Process in Linux?

A process is a running instance of a program. For example, when you execute:

```
python3 app.py
```

Linux creates a process to run the Python application.

Every process has its own Process ID (PID), which uniquely identifies it while it is running.

Common process information includes:

Term	Meaning
PID	Process ID
PPID	Parent Process ID
UID	User running the process
CPU	CPU resources consumed
Memory	Memory used by the process
State	Current process state

```
# View currently running processes
ps

# View processes with detailed information
ps aux

# View processes in a tree structure
pstree

# Check the current shell's PID
echo $$
```

2 Understanding Parent and Child Processes

Linux processes can create other processes.

- **Parent process:** The process that creates another process.
- **Child process:** The process created by a parent process.

For example, when you execute a command from a terminal, the shell may create a child process to execute that command.

```
# Check process IDs
ps -ef
```

Example output:

```
UID      PID   PPID  CMD
root       1      0  /sbin/init
john    1200   1100  bash
john    1250   1200  python3 app.py
```

Here:

- PID `1200` is the shell process.
- PID `1250` is the Python process.
- PPID `1200` means the Python process was created by the shell.

The first userspace process is commonly systemd, with PID `1`, on modern systemd-based Linux distributions.

3 Understanding Process States

A Linux process can exist in different states.

State	Meaning
R	Running or ready to run
S	Interruptible sleep
D	Uninterruptible sleep, commonly waiting for I/O
T	Stopped or traced
Z	Zombie process
I	Idle kernel thread

Check process states using `ps aux`. The process state appears in the `STAT` column.

What Is a Zombie Process?

A zombie is a process that has finished execution, but its parent has not yet collected its exit status. It remains as an entry in the process table.

```
# Find zombie processes and their parent PID (PPID)
ps -eo pid,ppid,stat,cmd | awk '$3 ~ /^Z/'
```

A zombie does not normally consume significant CPU or memory, but a large number of zombies may indicate a problem with the parent process.

> **IMPORTANT**
>
> You cannot fix a zombie simply by killing it because it has already exited. The parent process must collect its exit status, or the parent must terminate so that the process is adopted and reaped by another process.

What Is an Orphan Process?

An orphan is a process that is **still running** after its parent has exited. Linux automatically adopts it, usually by PID 1 (systemd), which collects its exit status when it finishes. Orphans are normal and usually harmless.

Zombie vs Orphan

Feature	Zombie	Orphan
Still running?	No, already exited	Yes
Parent	Alive, but has not collected exit status	Has exited
Uses CPU / memory	No, only a process table entry	Yes, like any running process
Cleaned up by	Parent collecting status, or parent exiting	Adopted and later reaped by PID 1
Concern	Many zombies indicate a parent bug	Usually normal

What Is a D-State Process?

A process in D state is waiting in uninterruptible sleep, often for disk or storage I/O.

Even `kill -9` may not terminate it immediately while it remains stuck in an uninterruptible kernel operation.

Investigate the underlying storage or I/O problem rather than repeatedly sending signals.

4 Monitoring Processes

Linux provides several commands to monitor running processes and resource usage.

ps

Displays a snapshot of process information.

```
# All processes, BSD style (shows %CPU and %MEM)
ps aux

# Find a specific process
ps aux | grep nginx

# All processes, full format (shows PPID)
ps -ef
```

top

Displays real-time process and system resource information.

```
top
```

Column	Meaning
PID	Process ID
USER	Process owner
%CPU	CPU usage
%MEM	Memory usage
TIME+	Total CPU time
COMMAND	Process command

Useful interactive keys inside `top`:

- `P`: Sort by CPU usage.
- `M`: Sort by memory usage.
- `k`: Send a signal to a process.
- `q`: Quit.

htop

An interactive alternative to `top`, with a more convenient interface. It may need to be installed first.

```
sudo apt install htop
htop
```

pgrep and pidof

```
# Find processes by name
pgrep nginx

# Show matching process names
pgrep -a nginx

# Find the PID of a running program
pidof nginx
```

Inspecting a Process Using /proc

Every running process has a directory under `/proc/PID` containing live information from the kernel.

```
# State, memory usage and thread count
cat /proc/1250/status

# Full command line used to start the process
cat /proc/1250/cmdline | tr '\0' ' '

# Number of open files
ls /proc/1250/fd | wc -l

# Open files and network connections
sudo lsof -p 1250
```

5 Managing Processes: kill and Signals

Linux uses signals to communicate with processes. The `kill` command sends a signal to a process. Despite its name, it does not always terminate the process.

Common Signals

Signal	Number	Purpose
SIGHUP	1	Hangup; often used to request configuration reload
SIGINT	2	Interrupt, commonly from Ctrl+C
SIGKILL	9	Forcefully terminate a process
SIGTERM	15	Request graceful termination
SIGSTOP	19	Stop a process
SIGCONT	18	Resume a stopped process

Signal numbers can vary on some architectures. Use `kill -l` to list all signals.

```
# Gracefully terminate a process (sends SIGTERM by default)
kill 1250

# Forcefully terminate a process
kill -9 1250

# Send a specific signal
kill -SIGTERM 1250

# Find and terminate processes by name
pkill nginx

# Match against the full command line
pkill -f "python3 app.py"
```

SIGKILL immediately terminates a process that can receive the signal. It should be a last resort because the application cannot perform normal cleanup.

Be careful with `pkill`: it may match multiple processes.

INTERVIEW QUESTION

Why should you not use `kill -9` first?

SIGKILL cannot be caught, so the application gets no chance to clean up. It may leave temporary files and lock files behind, lose data that was not yet written, drop active connections abruptly and leave child processes running. Always try SIGTERM first.

PRODUCTION RECOMMENDATION

First identify the correct process, attempt graceful termination, and investigate why it is not responding before using SIGKILL.

6 Foreground and Background Processes

A foreground process runs attached to the terminal. The terminal waits for it to finish or stop.

A background process runs without holding the terminal in the same way.

```
# Run a command in the background
python3 app.py &

# List background jobs started from the current shell
jobs

# Resume a stopped job in the background
bg

# Bring a job to the foreground
fg
```

The `&` symbol starts the command in the background. Press `Ctrl + Z` to suspend (stop) a foreground job.

nohup and disown

`nohup` allows a command to ignore the hangup signal, helping it continue when the terminal session closes.

```
nohup python3 app.py &
```

Output is normally written to `nohup.out` unless redirected.

If a job is already running, `disown` removes it from the shell's job list so it is not sent a hangup signal when the shell exits.

For long interactive work over SSH, tools such as `tmux` or `screen` keep sessions alive even if your connection drops.

> **IMPORTANT**
>
> Background execution and `nohup` are useful for temporary tasks, but they are not a replacement for a properly configured production service. Use systemd or another process supervisor for long-running applications.

7 Understanding systemd

`systemd` is an initialization and service management system used by many modern Linux distributions. It is commonly responsible for:

- Starting system services.
- Managing background services.
- Controlling service dependencies.

- Starting services during boot.
- Monitoring service status.
- Managing system startup targets.

The `systemctl` command is used to interact with systemd.

```
# Check whether systemd is running as PID 1
ps -p 1 -o comm=
```

What Is a Service?

A service is a background program managed by the operating system. Examples: Nginx, Apache, SSH, Docker, PostgreSQL, Redis.

8 Managing Services Using systemctl

```
# Check service status
sudo systemctl status nginx

# Start, stop and restart a service
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx

# Reload configuration without a full restart (if supported)
sudo systemctl reload nginx

# Enable or disable a service at boot
sudo systemctl enable nginx
sudo systemctl disable nginx

# Enable and start immediately
sudo systemctl enable --now nginx

# Check whether a service is enabled or active
systemctl is-enabled nginx
systemctl is-active nginx

# List running services
systemctl list-units --type=service --state=running
```

More Useful systemctl Commands

```
# List failed services
systemctl --failed

# Show the unit file systemd is actually using
systemctl cat nginx

# List all services and whether they are enabled
systemctl list-unit-files --type=service

# Completely block a service from starting
sudo systemctl mask nginx
sudo systemctl unmask nginx
```

Important Difference

Command	Meaning
`start`	Start service now
`stop`	Stop service now
`restart`	Stop and start service
`reload`	Reload configuration if supported
`enable`	Configure service to start at boot
`disable`	Prevent automatic startup at boot
`mask`	Block the service completely, even manual starts and dependencies

A service can be running but disabled at boot, or enabled at boot but currently stopped.

> INTERVIEW QUESTION
>
> **What is the difference between `disable` and `mask`?**
>
> `disable` only stops the service from starting automatically at boot; it can still be started manually or by another service. `mask` links the unit to `/dev/null`, so it cannot be started at all until it is unmasked.

9 Understanding systemd Service Files

Systemd services are commonly defined using unit files.

System-wide custom service files are usually stored in:

```
/etc/systemd/system/
```

Distribution-provided service files are commonly stored under:

```
/lib/systemd/system/
/usr/lib/systemd/system/
```

IMPORTANT

Files in `/etc/systemd/system/` take priority over package-provided files. Never edit package-provided unit files directly, because package updates overwrite them. Use `sudo systemctl edit nginx`, which creates a safe override file in `/etc/systemd/system/nginx.service.d/`.

Example Service File

```
[Unit]
Description=My Python Application
After=network.target

[Service]
User=appuser
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Understanding the Sections

Section	Purpose
`[Unit]`	Service description and dependencies
`[Service]`	How the application should run
`[Install]`	How the service is enabled

Important Settings

Setting	Purpose
`User`	User account used to run the application.
`WorkingDirectory`	Application working directory.
`ExecStart`	Command used to start the application.
`Restart`	Restart behavior after failure.
`After`	Startup ordering dependency.
`WantedBy`	Target used when enabling the service.

Useful Extra Settings

Setting	Purpose
`RestartSec=5`	Wait 5 seconds before restarting after a failure.
`Environment=APP_ENV=production`	Set an environment variable.
`EnvironmentFile=/etc/myapp/env`	Load environment variables from a file.
`LimitNOFILE=65535`	Maximum number of open files for the service.
`Type=simple`	Default; the `ExecStart` process is the main service process.

INTERVIEW TRAP

`After=network.target` only controls startup order; it does not wait until the network is fully ready. If the application needs a working network when it starts, use `Wants=network-online.target` and `After=network-online.target`.

Register and Start the Service

```
sudo systemctl daemon-reload
sudo systemctl enable --now myapp
sudo systemctl status myapp
```

After changing the service file, run `daemon-reload` so systemd reads the updated configuration.

10 Monitoring CPU and Memory Usage

A DevOps engineer must identify processes consuming excessive system resources.

```
# Check CPU and memory usage
top

# Check memory information
free -h
```

Example:

```
               total        used        free      shared  buff/cache   available
Mem:           7.7G        2.1G        1.2G       120M        4.4G        5.0G
Swap:          2.0G        100M        1.9G
```

Important fields:

Field	Meaning
`total`	Total available memory.
`used`	Memory currently in use.
`free`	Completely unused memory.
`buff/cache`	Memory used for buffers and cache.
`available`	Estimated memory available for new applications without swapping.

Linux uses otherwise-unused memory for caching, so low `free` memory alone does not necessarily indicate a problem.

```
# Check CPU information
lscpu

# Check the number of CPU cores
nproc

# Check load average
uptime
```

Example:

```
load average: 1.20, 1.50, 1.80
```

Load average represents the average number of tasks running or waiting for CPU and tasks in uninterruptible sleep over 1, 5 and 15 minutes.

Interpret it in relation to the number of CPU cores and workload. A load average of 4 may be normal on a system with many cores but concerning on a single-core system.

11 Process Priority and Resource Limits

nice and renice

Linux uses process priority to influence CPU scheduling. The nice value ranges from `-20` to `19`:

- `-20`: Higher scheduling priority.
- `0`: Default nice value.
- `19`: Lower scheduling priority.

```
# Start a command with lower scheduling priority
nice -n 10 python3 backup.py

# Change the nice value of an existing process
sudo renice 10 -p 1250

# Check process priority
ps -o pid,ni,pri,comm -p 1250
```

Only root can set a negative nice value or raise the priority of an existing process.

> **DEVOPS NOTE**
>
> Nice values influence CPU scheduling, but they do not impose strict CPU limits. For resource limits, Linux control groups (cgroups) and service-level resource controls are more appropriate.

Resource Limits: ulimit

`ulimit` controls per-process limits, such as the maximum number of open files or processes.

```
# Show all limits for the current shell
ulimit -a

# Show the maximum number of open files
ulimit -n

# Show the limits of a running process
cat /proc/1250/limits
```

- **Soft limit:** The limit currently enforced.
- **Hard limit:** The maximum value a user can raise the soft limit to.

Permanent limits for user login sessions are set in `/etc/security/limits.conf`:

```
appuser  soft  nofile  65535
appuser  hard  nofile  65535
```

> **INTERVIEW TRAP**
>
> `/etc/security/limits.conf` applies to login sessions. It does **not** apply to services started by systemd. For a systemd service, set `LimitNOFILE=` in the unit file, then run `daemon-reload` and restart the service.

12 Understanding Cron and Scheduled Tasks

Cron is used to schedule commands or scripts to run automatically at specified times. For example, a backup script may need to run every day at 2 AM.

```
# Edit the current user's cron jobs
crontab -e

# List scheduled jobs
crontab -l

# Remove the current user's cron jobs
crontab -r
```

> **WARNING**
>
> `crontab -r` deletes all your cron jobs immediately, without confirmation. The `r` key sits next to `e`, so this mistake is common. Back up first with `crontab -l > crontab-backup.txt`.

Cron Syntax

A cron entry contains five time fields followed by the command:

```
* * * * * command
| | | | |
| | | | └── Day of week (0-7, both 0 and 7 mean Sunday)
| | | └──── Month (1-12)
| | └────── Day of month (1-31)
| └──────── Hour (0-23)
└────────── Minute (0-59)
```

Common Examples

```
# Run a script every day at 2 AM
0 2 * * * /opt/scripts/backup.sh

# Run every five minutes
*/5 * * * * /opt/scripts/check.sh

# Run every Sunday at 3 AM
0 3 * * 0 /opt/scripts/weekly.sh

# Run on the first day of every month
0 0 1 * * /opt/scripts/monthly.sh

# Redirect output to a log file
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
```

`2>&1` redirects standard error to the same destination as standard output.

> **INTERVIEW TRAP**
>
> In a crontab, the `%` character has a special meaning (newline). A command such as `date +%F` breaks inside cron. Escape it as `date +\%F`, or move the command into a script.

System-Wide Cron

- `/etc/crontab` and files in `/etc/cron.d/` include an extra **username** field before the command.
- Scripts placed in `/etc/cron.daily/`, `/etc/cron.weekly/` and `/etc/cron.monthly/` run automatically at those intervals.

Important Cron Considerations

- Use absolute paths for scripts and commands.
- Ensure the script has execute permission.
- Set required environment variables explicitly; cron runs with a minimal environment and PATH.
- Redirect output so failures can be investigated.
- Verify that the cron service is running.

```
# Ubuntu: cron service and logs
sudo systemctl status cron
grep CRON /var/log/syslog

# RHEL-based: crond service and logs
sudo systemctl status crond
sudo tail /var/log/cron
```

systemd Timers (Modern Alternative)

On systemd-based systems, timers can replace cron jobs. A timer needs two files: a service that does the work and a timer that schedules it.

```
# /etc/systemd/system/backup.service
[Unit]
Description=Daily backup

[Service]
Type=oneshot
ExecStart=/opt/scripts/backup.sh

# /etc/systemd/system/backup.timer
[Unit]
Description=Run backup daily at 2 AM

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
systemctl list-timers
```

Benefits over cron: output goes to the journal automatically, `Persistent=true` runs a missed job after the server was down, and timers can use systemd dependencies.

13 Viewing Service Logs Using journalctl

When a service fails, logs are one of the first places to investigate. `journalctl` is used to view logs collected by systemd-journald.

```
# View all available journal entries
journalctl

# View logs for Nginx
sudo journalctl -u nginx

# View the last 50 lines
sudo journalctl -u nginx -n 50

# Follow logs in real time
sudo journalctl -u nginx -f

# View logs since a specific time
sudo journalctl -u nginx --since "1 hour ago"

# View logs from the current boot
journalctl -b

# View logs from the previous boot
journalctl -b -1
```

More Useful journalctl Options

```
# Show only errors and more severe messages
sudo journalctl -u nginx -p err

# Show logs within a time range
sudo journalctl --since "2026-11-05 02:00" --until "2026-11-05 03:00"

# Show kernel messages
journalctl -k

# Check journal disk usage and clean it up
journalctl --disk-usage
sudo journalctl --vacuum-size=500M
```

IMPORTANT

On some systems the journal is stored only in memory (`/run/log/journal`), so logs are lost after a reboot and `journalctl -b -1` shows nothing. To keep logs across reboots, set `Storage=persistent` in `/etc/systemd/journald.conf` and restart `systemd-journald` .

DEVOPS NOTE

When a service fails to start, combine `systemctl status` and `journalctl -u service-name` to identify the reason.

14 Practical DevOps Troubleshooting Scenarios

Scenario 1 Nginx service is not starting

```
# Check service status
sudo systemctl status nginx

# Check detailed logs
sudo journalctl -u nginx -n 50

# Check Nginx configuration syntax
sudo nginx -t
```

If the configuration is valid, investigate port conflicts, file permissions and other dependencies.

Scenario 2 Application is consuming high CPU

Find the process using `top` , and sort by CPU usage using `P` .

```
# Find the process details
ps -fp 1250

# See which threads inside the process are using CPU
top -H -p 1250
```

Check its command and logs before deciding whether to restart or terminate it.

Scenario 3 Server memory usage is high

```
# Check memory
free -h

# Find processes consuming the most memory
ps aux --sort=-%mem | head

# Check whether the OOM killer has stopped any process
sudo dmesg -T | grep -i "killed process"
```

Investigate whether the memory usage is expected, whether the application has a memory leak, or whether the server needs additional capacity.

Scenario 4 Application process is running but website is unavailable

```
# Check the process
ps aux | grep myapp

# Check service status
sudo systemctl status myapp

# Check whether the application is listening on its expected port
sudo ss -lntp

# Test the application locally
curl -I http://localhost:8080

# Check application logs
sudo journalctl -u myapp -n 50
```

If `ss` shows the application listening on `127.0.0.1`, it accepts only local connections. It must listen on `0.0.0.0` (or the server's IP) to be reachable from outside.

A running process does not automatically mean the application is healthy or reachable.

Scenario 5 A background process stops after closing the terminal

A process started directly in a terminal may receive a hangup signal when the session ends. For temporary tasks, use:

```
nohup python3 app.py > app.log 2>&1 &
```

For production applications, configure a systemd service instead.

Scenario 6 Application logs show "Too many open files"

```
# Check the limit of the running process
cat /proc/1250/limits | grep "open files"

# Count the files it currently has open
ls /proc/1250/fd | wc -l
```

If the process is near its limit, first check for a file or connection leak in the application. If the application genuinely needs more, raise `LimitNOFILE=` in its systemd unit file, then run `daemon-reload` and restart the service.

Scenario 7 Service keeps restarting in a loop

```
# Status shows "activating (auto-restart)" and a restart counter
sudo systemctl status myapp

# Find the reason it keeps crashing
sudo journalctl -u myapp -n 100
```

If systemd reports *"Start request repeated too quickly"*, it has stopped retrying. Fix the root cause, then run `sudo systemctl reset-failed myapp` and start the service again.

15 Common Interview Questions

Q1. What is a process in Linux?

A process is a running instance of a program. Every process has a unique PID while it exists.

Q2. What is the difference between PID and PPID?

PID identifies a process. PPID identifies its parent process.

Q3. What is the difference between `ps` and `top`?

`ps` provides a snapshot of process information. `top` provides continuously updated process and resource information.

Q4. What is a zombie process?

A zombie is a process that has exited but whose parent has not yet collected its exit status.

Q5. What is the difference between SIGTERM and SIGKILL?

SIGTERM requests graceful termination. SIGKILL forcefully terminates a process and cannot be caught or ignored by it.

Q6. What is the difference between foreground and background processes?

A foreground process is attached to the terminal. A background process runs without holding the terminal in the same way.

Q7. What is nohup used for?

It allows a command to ignore hangup signals, helping it continue after the terminal session closes.

Q8. What is systemd?

Systemd is a system and service manager responsible for initializing the system and managing services on many modern Linux distributions.

Q9. What is the difference between `start` and `enable` in systemctl?

`start` runs a service now. `enable` configures it to start automatically during boot.

Q10. What is the difference between restart and reload?

Restart stops and starts the service. Reload asks the service to apply configuration changes without a full restart, if supported.

Q11. How do you check why a service failed?

```
systemctl status service-name
journalctl -u service-name -n 50
```

Q12. What is load average?

It represents the average number of tasks running or waiting for CPU and tasks in uninterruptible sleep over 1, 5 and 15 minutes.

Q13. How do you find the process consuming the most memory?

```
ps aux --sort=-%mem | head
```

Q14. What is cron?

Cron is a scheduler used to execute commands or scripts automatically at specified times.

Q15. What is the difference between a process and a service?

A process is a running instance of a program. A service is a background application or system component, commonly managed by a service manager such as systemd.

Q16. What is the difference between a zombie and an orphan process?

A zombie has already exited but its parent has not collected its exit status. An orphan is still running after its parent has exited, and it is adopted by PID 1.

Q17. How do you get rid of zombie processes?

Killing the zombie has no effect because it has already exited. Find its parent using the PPID, then fix or restart the parent process. Once the parent collects the exit status or exits, the zombies are cleaned up.

Q18. What is the difference between `systemctl disable` and `systemctl mask`?

`disable` prevents automatic startup at boot, but the service can still be started manually. `mask` blocks the service from starting in any way until it is unmasked.

Q19. You increased limits in `/etc/security/limits.conf`, but a service still shows "Too many open files". Why?

`limits.conf` applies to login sessions, not to systemd services. Set `LimitNOFILE=` in the service's unit file, run `daemon-reload` and restart the service.

16 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ Linux processes, PIDs and parent-child relationships.
- ✓ Process states, including zombie, orphan and D-state processes.
- ✓ Monitoring using `ps`, `top`, `htop` and `/proc`.
- ✓ Process control using `kill`, signals and `pkill`.
- ✓ Foreground and background execution.
- ✓ systemd and service management, including `mask` and drop-in overrides.

- ✓ Creating and managing systemd service files.
- ✓ CPU, memory and load average monitoring.
- ✓ Process priority using `nice` and `renice`.
- ✓ Resource limits using `ulimit` and `LimitNOFILE`.
- ✓ Scheduling tasks using cron and systemd timers.
- ✓ Investigating service logs using `journalctl`.
- ✓ Troubleshooting failed services and resource-intensive processes.

PRACTICAL EXERCISES

1. Run `ps aux` and identify five running processes.
2. Find the PID of Nginx or another running service.
3. Open `top` and identify the highest CPU-consuming process.
4. Start a test command in the background and inspect it using `jobs`.
5. Practice sending SIGTERM to a test process.
6. Install Nginx and manage it using `systemctl`.
7. Enable Nginx to start automatically at boot.
8. Create a simple custom systemd service for a test script.
9. View the service logs using `journalctl`.
10. Create a cron job that writes the current date and time to a log file every five minutes.
11. Mask a test service, try to start it, then unmask it.
12. Check the open-file limit of a running process using `/proc/PID/limits`.

NEXT CHAPTER

Chapter 06: Linux Storage and Filesystem Management

You will learn disk partitions, filesystems, mounting, `/etc/fstab`, disk usage, LVM, swap, and how to troubleshoot disk-space and filesystem problems in production servers.

END OF CHAPTER 5